

Timings Table:

	Fast + randomized	Walking + randomized
10000 points	4.4526 secs	5.1491 secs
100000 points	53.8490 secs	136.1317 secs
1000000 points	669.1580 secs	DNF within 30 mins

Question: Why do you think the fastest algorithm is fastest?**Answer:**

The History DAG is faster because it replaces a local topological walk with a global hierarchical search. The walking method is restricted to neighboring triangles and must cross a linear path of $O(\sqrt{N})$ edges to locate a point, effectively re-scanning the mesh with every insertion. On the other hand, the History DAG preserves the evolution of the triangulation as a search tree. This allows the algorithm to start at the largest triangle and move through levels of overlapping child nodes, discarding large regions of the mesh at each step. This reduces the search depth to $O(\log N)$, ensuring that the number of geometric tests grows only slightly even as the dataset size doubles. We can see this in action at 1 million points, where the walking method takes too long compared to the history DAG.

Through profiling with cProfile, I found that for 10,000 points, the walking method required 2,146,529 calls to the `right_of` predicate because it must navigate $O(\sqrt{N})$ triangles per insertion. On the other hand, the History DAG method reduced these calls to just 120,182, a 94% reduction, by utilizing an $O(\log N)$ tree search.

Question: Does failing to randomize the order of the vertices significantly change the running time of the incremental insertion algorithm with the fast point location method?**Answer:**

Test Dataset: `grid.node` (25 vertices in a structured 5 x 5 grid).

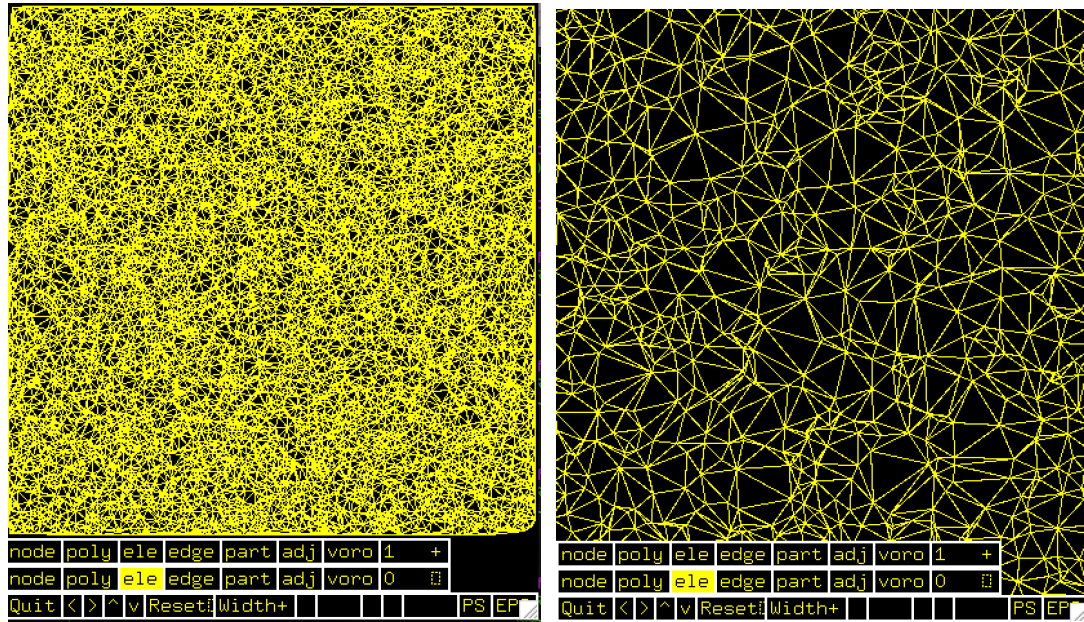
At the small scale of 25 points, the difference was minimal (0.0127s ordered vs. 0.0100s randomized). However, randomization is essential for the History DAG's stability.

In the ordered grid test, the points are collinear at the start which we can see since the first five points share $y=0$. This forces the algorithm to handle degenerate cases sequentially. Randomized insertion ensures the triangulation grows uniformly in all directions, maintaining the $O(\log N)$ search depth guarantee and providing a 20% speedup even at this tiny scale.

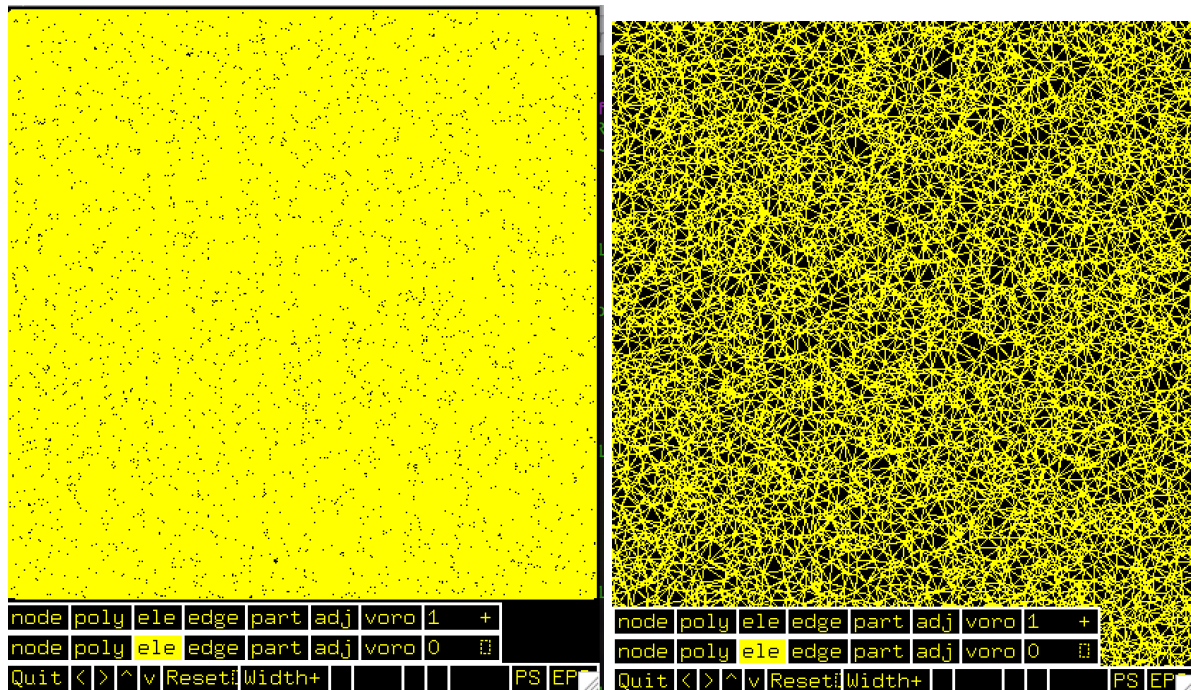
Code Credits:

- Jonathan Shewchuk's predicates file
- Primitives for the manipulation of general subdivisions and the computation of Voronoi Pseudocode

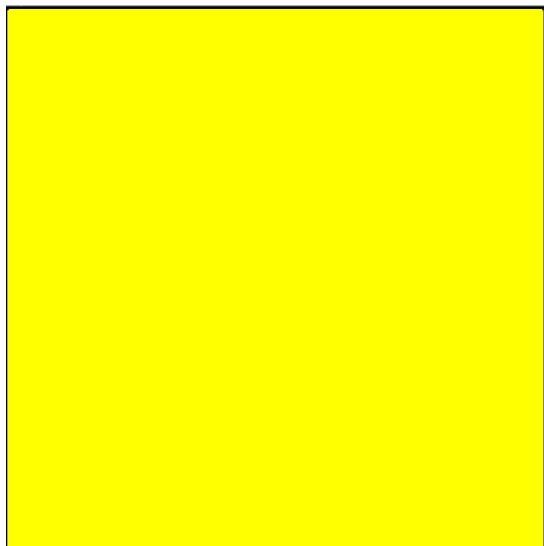
Show Me Visualization for 10000 points:



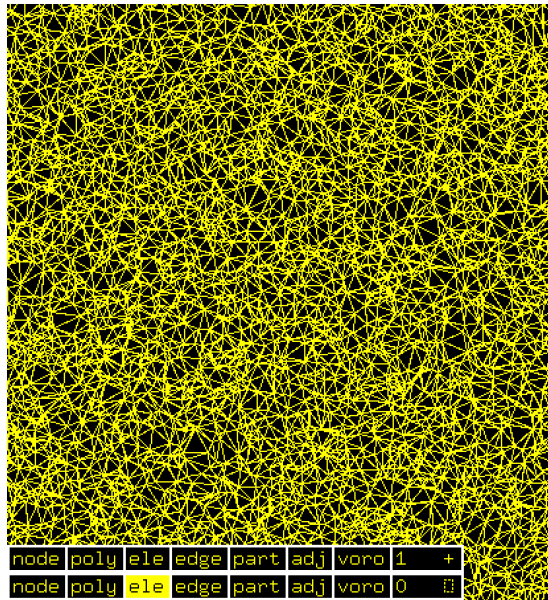
Show Me Visualization for 100000 points:



Show Me Visualization for 1000000 points:



node	poly	ele	edge	part	adj	voro	1	+
node	poly	ele	edge	part	adj	voro	0	⌂
Quit < > ^ v Reset Width+ PS EF								



node	poly	ele	edge	part	adj	voro	1	+
node	poly	ele	edge	part	adj	voro	0	⌂
Quit < > ^ v Reset Width+ PS EF								